

Firefox Add-on SDK released

The first version of Firefox Add-on SDK, which allows developers to create Firefox add-ons using web technologies, has been released

CSS 2.1 now a standard

After many years of work, CSS 2.1 has been standardised, along with the CSS3 Color module



After “transition-property” we have a comma-separated list of properties that we wish to animate. After “transition-duration”, we’re providing the transition durations for each specified property in the same order as the properties. If we had specified just one duration, it would be used for all properties.

There are two more properties of transitions:

- **transition-delay:** You can use this to specify a delay before the animation is triggered.
- **transition-timing-function:** This function is used to calculate intermediate values between the two transition points. Predefined ones are ease, linear, ease-in, ease-out, ease-in-out. Or you can specify your own via cubic-bezier (a, b, c, d) where a, b, c, d are numbers. Transitions need not be triggered only on hover. If the CSS styles of an element change under other conditions (using JavaScript), clicking for example, they will be animated if a transition is specified.

CSS animations

Transitions and Animations share a lot of similar concepts, so let’s first look at the common elements. Just like transitions, animations have a duration (“animation-duration”), a delay (“animation-delay”) and a timing function (“animation-timing-function”).

Animations have two components: first, a description of the animation and second, a set of keyframes that define how the animation occurs over time.

Animations are executed immediately on all elements matching their CSS selector. This might not be desirable if (for example) you wish to apply an animation when someone clicks on a Play button. For that, you’ll need to use JavaScript to apply the animated class when required.

Here’s how you can specify an animation on a div with an id of “animated”:

```
CSS:
#animated {
  animation-name: test-animat-
tion;
  animation-duration: 4s;
  animation-delay: 3s;
  animation-iteration-count:
10;
  animation-direction: alter-
```

VENDOR PREFIXES IN CSS

A standard is only useful if it’s implemented. However, standards are often implemented even before they’re fully defined. Sometimes, there are differences in how they’re defined in different browsers. As such, new CSS features under development or in the process of standardisation are implemented with vendor prefixes.

Each browser engine uses a different prefix: Mozilla’s Gecko engine in Firefox uses a prefix of “-moz-”, Opera uses the “-o-” prefix, and Microsoft’s Internet Explorer uses the “-ms-” prefix. Since Safari and Chrome use the same engine (WebKit), their prefix is also the same “-webkit-”. Initially, to use the same CSS feature on multiple browsers, you’ll need to define the same property multiple times – once for each browser. However, eventually the prefix will be dropped, and you’ll be able to use the same feature on all browsers without a prefix. Since prefixes will be eventually dropped, it’s a good idea to define the property without a prefix as well, to ensure it continues working even when the prefix is dropped.

```
name;
}
```

Here we’re specifying an animation name, duration of four seconds, and a delay of three seconds. Via the iteration count we’re specifying that the animation has to run 10 times. The animation-direction property is set to “alternate” to specify that the animation must reverse when completed. Thus, the animation will go forward, and in the subsequent iteration will play backwards, and so on.

This, of course, has said nothing of what the animation itself will look like. That we specify using the “@keyframes” keyword as follows:

```
CSS:
@keyframes test-animation {
  from {
    width: 10px;
    height: 4px;
  }
  25% {
    width: 50px;
    height: 4px;
  }
  to {
    width: 50px;
    height: 200px;
  }
}
```

That is, @keyframes followed by the name of the animation, here “test-animation”.

These set of keyframes will accomplish the following:

- In the beginning, the height and width of the div will be set to four pixels, and 10 pixels respectively.
- From the beginning to 25 percent of the way through the animation, i.e. after 1 second, the width will increase to 50 pixels, but the height will remain the same.

- From 25 percent till the end of the animation, the width will remain the same, but the height will increase to 200 pixels. Now let’s create a small animation of a red square moving between two points:

```
CSS:
#animated {
  animation-name: test-animat-
tion;
  animation-duration: 4s;
  animation-iteration-count:
10;
  animation-direction: alter-
nate;
  animation-timing-function:
linear;
  background-color: red;
  border: thin solid black;
  width: 20px;
  height 20px;
  position: absolute;
}
@keyframes test-animation {
  from {
    left: 0px;
  }
  50% {
    left: 300px;
    width: 50px;
  }
  to {
    left: 600px;
  }
}
```

This will bounce the square between 0px and 600px, while increasing the width to 50px in the middle.

CSS Animation like transitions, is still developing as a standard, so in all examples we’ve given here you should add a prefix, for each browser. You can use them on WebKit browsers and Firefox.

Animations are used in abundance on the web, and it’s only a matter of time before